

Week14: 期末备考

Updated 2026-06-02 14:13 GMT+8
Compiled by Hongfei Yan (2026 spring)

机考策略与时间分配建议

同学们，明天的机考将精选出 **6 道题**（包含 **2 道 Easy**、**3 道 Medium**、**1 道 Tough**）。

上机考试不仅考察算法的掌握，还考验时间管理与心态调整。为了帮助大家在考场上稳定发挥，这里整理了一份实用的策略建议，供大家参考：

一、核心答题策略

1. 先易后难，稳扎稳打

- **迅速破冰**：开考后建议先快速浏览题目。优先解答 **2 道 Easy 题**，它们通常逻辑直接、代码量小，能帮你迅速拿到基本分，建立答题信心，平复紧张情绪。
- **攻坚中档题**：**3 道 Medium 题**是本次考试的得分核心，也是拉开差距的关键。在做完简单题后，应将主要精力和时间放在这三道题上，稳步推进。

2. 合理止损，切忌“钻牛角尖”

- **思考止损（5分钟原则）**：如果读完一道题，仔细思考了 5 分钟依然完全没有思路、或者看不懂输入输出格式，请**果断先跳过并做好标记**。不要在空想中消耗宝贵的考试时间。
- **调试止损（15分钟原则）**：如果某道题你已经写好了代码，但调试了 15 分钟依然在报 WA（答案错误）、RE（运行时错误）或 TLE（超时），建议先保存好当前代码，**换换脑子去做别的题**。回过头来重新审视，往往能一眼看出之前的盲点。

3. 注意细节，稳拿基本分

- **I/O 格式**：请务必注意题目要求的输入输出格式（如空格、换行、大小写拼写等），避免因细节失分。
- **边界情况**：在提交前，多想一想极限情况（如 $N = 1$ 、数据为空、负数、极大值等）。

4. 保持心态从容

- 进入考场若感到紧张，可以尝试深呼吸（吸气4秒，屏息7秒，呼气8秒）来舒缓压力。
- 记住“我难人难，我不畏难；我易人易，我不大意”。专注自己的电脑屏幕和代码，不受周围敲击键盘速度的影响。

二、时间分配参考表（以 112 分钟为例）

以下是一套比较合理的时间控制方案，大家可以根据自己的做题节奏灵活微调：

题目类型	题量	建议单题耗时	累计建议时间	策略重点
Easy 简单题	2 道	5 ~ 10 分钟	开考后 20 分钟内完成	快速写完并检查，确保一次性通过。
Medium 中等题	3 道	15 ~ 20 分钟	耗时约 45 ~ 60 分钟	得分核心，理清思路再动手，注意边界情况。
Tough 挑战题	1 道	30 ~ 50 分钟	放在最后攻坚	此时已有 5 题打底，心态放平。
机动调试时间	-	-	最后 5 ~ 10 分钟	用于处理意外 TLE、格式不符或整体查漏补缺。

由于 112 分钟时间稍显紧张，万一某道 Medium 题卡了壳，千万不要慌张。立刻启用“15分钟止损原则”，先跳过去看下一道。保证在第 80 分钟时，手上有 4~5 道题已经提交通过，这样最后的心态会非常轻松。

祝大家明天机考顺利，写出的代码全部 One-Pass AC！

数据B-15班 期末机考通知

cs201 2026spring Final Exam, <http://cs101.openjudge.cn/cs2012026springclass15/>

公告：

你好，OpenJudge CS101，

我很高兴邀请大家参加 2026spring-cs201: DS Algo DS-15班 课程期末机考。比赛将于 2026 年 6 月 3 日星期三 15:08 (UTC+8) 举行。我们准备了 6+2 道题目，参赛者需在 112 分钟内完成。

比赛采用 ICPC 赛制，每道题目没有部分分数，可以无限次提交。比赛全部为传统题，只提供中文题面。

比赛期间，请参赛者只访问 OpenJudge，不能访问其他网站，不要查看 OpenJudge 考试之前自己提交的代码。

允许携带 10 张 A4 纸大小的 cheat sheet，以及空白草稿纸，但不允许参考电子媒介。

题目编号前面的大写字母，相应表明是 Easy/Medium/Tough/Ultra Tough/Impossible 级别。U 开始题目不计分。

如果题目中存在任何问题，请及时向助教或者老师提问以获得解释。

感谢大家参加。祝大家好运，并且玩的开心。

GMyhf

其它注意事项：

登录别人的账号即视为违纪甚至作弊。把自己的账号密码告诉别人，被别人登录，也视为违纪甚至作弊。如果考前别人用过你的账号，请立即修改密码。

请确保你的昵称格式为 24nxxxxx，后面部分是学号。<http://cs101.openjudge.cn/mine>

有同学昵称22n, 21n, 20n, 19n开始也是可以的，学号别错，就能找到你的成绩。

考试时间：2026年6月3日周三 15:08 - 17:00

考试地点：

-5号机房（理科1号楼二层1203W）

-6A号机房（理科1号楼二层1250W）、6B号机房（1249W）

提交程序语言选项：

-Python3, PyPy3, G++

需要携带的物品：

-10页A4 cheat sheet

-空白草稿纸（页数不限，但建议适量，应该也用不到很多张，部分题目可能需要推导公式之类的）

-笔

可以携带的物品：
-按键声音较小，没有存储和接发信息功能的键盘和鼠标

考试中不允许使用有存储和发送信息功能的电子设备，只允许使用机房电脑进行答题

注意事项：

- 按照 机房位置安排-数算B-15班-20260603.xls，找到自己座位，重启电脑登录自己的门户账号，登录网关（its.pku.edu.cn），打开<http://cs101.openjudge.cn/> 准备考试
- 请记得按照考试说明更新自己openjudge的账户名称，把你的昵称格式为 25nxxxxx，后面部分是学号。
- 有同学昵称24n, 23n, ..., 19n开始也是可以的，学号别错，才能找到你的成绩。
- 关于PyPy3：请注意，尽管我们支持PyPy3作为提交选项之一，但其兼容性可能不如Python3稳定，尤其是在处理较为简单的题目时可能会遇到问题。因此，建议大家优先使用Python3进行尝试，确保代码能够顺利通过测试。

考题核心算法与数据结构

这些题目涵盖了数据结构、图论、动态规划、高级算法等多个领域。为了便于复习和对比，以下将这些题目按照核心算法与数据结构进行了分类整理：

一、基础数据结构与双指针（线性表、栈、队列）

这类题目主要考察对基础数据结构（如栈、队列、哈希表）的特性运用，以及双指针、滑动窗口的区间控制能力。

题号 & 题目名称	难度	核心算法 / 数据结构	时间复杂度	核心解题思路简述
	基础	哈希表 (set)	$O(N)$	
	基础	双指针 / 单次遍历	$O(N)$	
	基础	队列 (deque)	$O(N)$	
	基础	哈希表 / 二分查找	$O(N + Q)$	
	中等	滑动窗口 / 双指针	$O(N + M)$	
	中等	单调栈	$O(N)$	
	困难	栈 / 字符串处理	$O(N)$	
	实用	栈 / 状态历史树	$O(Q)$	

二、贪心、排序与二分

在特定规律下，通过排序、局部最优（贪心）或折半搜索来降低问题规模。

题号 & 题目名称	难度	核心算法 / 数据结构	时间复杂度	核心解题思路简述
	基础	排序	$O(N \log N)$	
	中等	贪心算法	$O(N)$	
	困难	堆	$O(N \log N)$	
	困难	二分	$O(N \cdot 2^{N/2})$	

三、图论与拓扑结构（连通性、缩点、依赖关系）

图论题目涉及并查集、有向无环图（DAG）的拓扑排序，以及强连通分量（SCC）的缩点应用。

题号 & 题目名称	难度	核心算法 / 数据结构	时间复杂度	核心解题思路简述
	中等	并查集 (DSU)	$O(N \cdot \alpha(N))$	
	中等	树 / 深度优先搜索 (DFS)	$O(N \log(\deg))$	
	中等	拓扑排序 (Kahn 算法)	$O(M + E)$	
	困难	强连通分量 (Tarjan) / DAG	$O(N + M)$	
	困难	SCC (Kosaraju/Tarjan) / 拓扑排序 / DP	$O(N + M)$	

四、动态规划与状态压缩

通过状态转移方程，利用历史子问题的最优解来推导当前问题的最优解。

题号 & 题目名称	难度	核心算法 / 数据结构	时间复杂度	核心解题思路简述
	基础	线性动态规划	$O(N)$	
	中等	DP + 滑动窗口	$O(N)$	
	中等	DFS序 / 树上背包 DP	$O(N \cdot K)$	
	实用	构造 / 动态规划	视方案而定	
	实用	带状态 BFS / 状态压缩 DP	$O(K \cdot N^2 \cdot C + 2^K \cdot K^2)$	

五、高级数据结构（线段树、单调队列等）与数学搜索

解决超大规模数据（如 $N \geq 10^5$ ）下的区间最值更新与数论计算问题。

题号 & 题目名称	难度	核心算法 / 数据结构	时间复杂度	核心解题思路简述
	中等	滑动窗口 / 单调队列	$O(N)$	
	中等	剪枝搜索 / 约数分解	$O(Q \cdot \text{DFS})$	
	困难	数论 (最小质因数) / BFS	$O(M \log \log M + N \cdot \omega)$	
	实用	离散化 / 树状数组 (BIT) / 懒删除堆	$O(N \log N)$	
	实用	线段树 / 懒惰传播 (Lazy Propagation)	$O(Q \log N)$	

机考大纲

汇总课程大纲。不包括超纲内容：树状数组、线段树、SMT（斯坦纳树）、AVL、KD Tree、倍增法（LCA的高效算法），其他课件内容应该都在出题范围内。虽然超纲内容不用掌握，但是有的题目多解，例如：可能套树状数组或者线段树就简单了。另外，机考不考KMP、马拉车算法。

根据课程资料，以下是整理后的课程大纲汇总。本大纲已剔除超纲内容（树状数组、线段树、最小斯坦纳树、AVL树、KD树、倍增法），重点聚焦于核心考点与基础数据结构：

一、算法基础与面向对象编程 (OOP)

- 算法复杂度分析：
 - 渐近记号：掌握 O （上界/最坏情况）、 Ω （下界）、 Θ （紧确界/平均情况）的数学定义。
 - RAM模型：假设基本算术和逻辑运算为常数时间。
 - 常见复杂度级别： $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$ 。
- Python OOP 实现：
 - 类与对象：理解类作为抽象数据类型（ADT）的实现载体。
 - 魔术方法：掌握 `__init__`（构造）、`__str__`（字符串表示）、`__add__`（运算符重载）等。
 - 封装、继承与多态：利用继承实现代码复用（如 `DFSGraph` 继承 `Graph`）。

二、线性结构 (Linear Structures)

- 顺序存储 vs 链式存储：
 - 数组/列表：支持 $O(1)$ 随机访问，插入删除通常为 $O(n)$ 。
 - 链表 (Linked List)：单向、双向及循环链表。掌握快慢指针技巧（找中点、判环）。
- 栈 (Stack)：
 - 特性：后进先出 (LIFO)。
 - 典型应用：括号匹配、进制转换、中缀转后缀（调度场算法）及后缀表达式求值。
 - 高级技巧：单调栈（寻找下一个更大/更小元素）。
- 队列 (Queue)：
 - 特性：先进先出 (FIFO)。
 - 变体：双端队列 (Deque) 及循环队列。
 - 典型应用：约瑟夫问题、打印机模拟任务。
 - 高级技巧：单调队列（滑动窗口最大值）。

三、字符串与位运算优化

- KMP 算法：
 - 核心：计算最长相等前后缀（LPS）表，实现主串指针不回退的 $O(n + m)$ 匹配。
 - 性质：利用 LPS 判定字符串的最小循环节。
- 位运算 (Bit Manipulation)：
 - 基础操作：`&`, `|`, `^`, `~`, `<<`, `>>`。
 - 常用技巧：`lowbit(x) = x & -x`、判断 2 的幂 `x & (x-1) == 0`。

- 状态压缩：利用二进制位表示集合及其运算（并、交、差）。
- 前缀和 (Prefix Sum):
 - 1D/2D 前缀和：通过 $O(n)$ 预处理实现 $O(1)$ 区域求和查询。

四、排序算法

- 基础排序 ($O(n^2)$): 冒泡排序、选择排序、插入排序。
- 进阶排序 ($O(n \log n)$):
 - 快速排序：分治法，注意 Pivot 选择及不稳定性。
 - 归并排序：分治法，稳定排序，常用于统计逆序对。
 - 堆排序：利用二叉堆实现。
- 希尔排序：插入排序的改进版。

五、树形结构 (Tree)

- 二叉树基础:
 - 性质：叶子节点数 $n_0 = n_2 + 1$ ；完全二叉树的高度及节点关系。
 - 遍历：前序、中序、后序（递归与迭代实现）、层序遍历 (BFS)。
 - 二叉搜索树 (BST)：左小右大性质，中序遍历为升序。
- 多叉树：长子-兄弟表示法（将多叉树转换为二叉树结构）。
- 二叉堆与优先级队列:
 - 实现：利用完全二叉树的数组表示（父节点 $(i-1)/2$ ）。
 - 操作：上浮 (perc_up) 与下沉 (perc_down)，建堆复杂度为 $O(n)$ 。
- 哈夫曼编码 (Huffman):
 - 原理：贪心策略，构造带权路径长度 (WPL) 最小的扩充二叉树。
- 并查集 (DSU):
 - 核心操作：查找 (Find) 与合并 (Union)。
 - 优化：路径压缩与按秩/大小合并，均摊复杂度接近 $O(1)$ 。

六、图论算法 (Graph)

- 图的表示：邻接矩阵（适合稠密图）与邻接表（适合稀疏图）。
- 图的遍历:
 - BFS：求无权图最短路径。
 - DFS：掌握发现时间 d 与结束时间 f 。
- 拓扑排序 (Topological Sort):
 - 算法：Kahn算法 (基于入度/BFS) 或 DFS 结束时间逆序。
- 强连通分量 (SCC):
 - 算法：Kosaraju (两次DFS) 或 Tarjan 算法 (单次DFS+栈)。
- 最小生成树 (MST):
 - Prim 算法：归并顶点，适合稠密图。
 - Kruskal 算法：按边排序+并查集，适合稀疏图。
- 最短路径 (Shortest Paths):

- **Dijkstra**: 单源、非负权图的最短路。
- **Bellman-Ford / SPFA**: 支持负权边, 检测负环。
- **Floyd-Warshall**: 全源最短路, 基于动态规划。
- **关键路径 (AOE网)**:
 - **核心**: 计算事件最早/最晚发生时间, 确定松弛时间为0的关键活动。

昨天群里讨论题目

T1192.查找集群内的关键连接

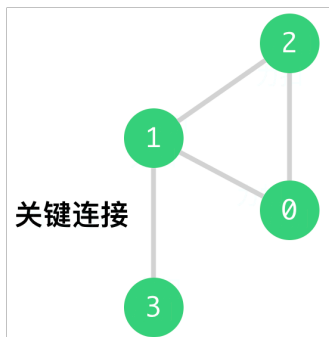
BCC双连通分量, <https://leetcode.cn/problems/critical-connections-in-a-network/>

力扣数据中心有 n 台服务器, 分别按从 0 到 $n-1$ 的方式进行了编号。它们之间以 **服务器到服务器** 的形式相互连接组成了一个内部集群, 连接是无向的。用 `connections` 表示集群网络, `connections[i] = [a, b]` 表示服务器 `a` 和 `b` 之间形成连接。任何服务器都可以直接或者间接地通过网络到达任何其他服务器。

关键连接 是在该集群中的重要连接, 假如我们将它移除, 便会导致某些服务器无法访问其他服务器。

请你以任意顺序返回该集群内的所有 **关键连接**。

示例 1:



```
1 输入: n = 4, connections = [[0,1],[1,2],[2,0],[1,3]]
2 输出: [[1,3]]
3 解释: [[3,1]] 也是正确的。
```

示例 2:

```
1 输入: n = 2, connections = [[0,1]]
2 输出: [[0,1]]
```

提示:

- $2 \leq n \leq 105$
- $n - 1 \leq \text{connections.length} \leq 10^5$
- $0 \leq a_i, b_i \leq n - 1$
- $a_i \neq b_i$

- 不存在重复的连接

这道题要求寻找无向连通图中的所有“关键连接”（即桥）。在一个连通图中，如果删除某条边会使得图不再连通，那么这条边就是桥（Bridge）。

我们可以使用 **Tarjan 算法**（基于深度优先搜索 DFS）来高效地寻找所有的桥。

算法思路

在 DFS 遍历的过程中，我们为每个节点 u 记录两个关键的值：

1. `dfn[u]`：节点 u 被深度优先搜索到的次序号（时间戳）。
2. `low[u]`：节点 u 或者是 u 的子树中的节点，通过一条非树边（回边）能够到达的最小时间戳。

当我们在 DFS 树中从节点 u 访问到节点 v 时（且 v 不是 u 的父节点）：

- 如果 v 还没有被访问过，我们递归地对 v 进行 DFS。在 v 的 DFS 结束后，我们用 `low[v]` 更新 `low[u]`（即 `low[u] = min(low[u], low[v])`）。
 - 此时，如果满足 `low[v] > dfn[u]`，说明从 v 及其子树出发，没有任何一条路径可以不通过边 (u, v) 访问到 u 或其祖先。因此，边 (u, v) 就是一个关键连接（桥）。
- 如果 v 已经被访问过，说明找到了一个环，我们用 `dfn[v]` 更新 `low[u]`（即 `low[u] = min(low[u], dfn[v])`）。

Python 3 实现

由于节点的数量最多为 10^5 ，递归深度可能会超过 Python 默认的限制，我们需要使用 `sys.setrecursionlimit` 来调大递归深度。

```
1 import sys
2 from typing import List
3
4 # 调大 Python 的最大递归深度以防止栈溢出
5 sys.setrecursionlimit(200000)
6
7 class Solution:
8     def criticalConnections(self, n: int, connections: List[List[int]]) -> List[List[int]]:
9         # 1. 构建邻接表
10         adj = [[] for _ in range(n)]
11         for u, v in connections:
12             adj[u].append(v)
13             adj[v].append(u)
14
15         dfn = [-1] * n
16         low = [-1] * n
17         bridges = []
18         time = 0
19
20         # 2. 定义 DFS 函数
21         def dfs(u: int, parent: int):
22             nonlocal time
23             dfn[u] = low[u] = time
24             time += 1
25
26             for v in adj[u]:
27                 if v == parent:
28                     continue
29                 if dfn[v] == -1:
30                     # v 未被访问，递归访问
```



```

31         dfs(v, u)
32         # 更新当前节点的 low 值
33         if low[v] < low[u]:
34             low[u] = low[v]
35         # 判断是否为桥
36         if low[v] > dfn[u]:
37             bridges.append([u, v])
38     else:
39         # v 已被访问，且不是父节点，更新当前节点的 low 值
40         if dfn[v] < low[u]:
41             low[u] = dfn[v]
42
43     # 因为整个网络是连通的，所以从任意节点（例如 0）开始 DFS 即可遍历所有的点
44     dfs(0, -1)
45     return bridges

```

复杂度分析

- **时间复杂度：** $O(V + E)$ ，其中 $V = n$ 是服务器的数量， E 是连接的数量。DFS 过程中每个顶点和每条边都只被访问了常数次数。
- **空间复杂度：** $O(V + E)$ 。邻接表需要 $O(V + E)$ 的空间，`dfn` 和 `low` 数组以及递归调用栈最多需要 $O(V)$ 的空间。

P1991 无线通讯网（黄色 普及/提高-）

MST变种，<https://www.luogu.com.cn/problem/P1991>

国防部计划用无线网络连接若干个边防哨所。2 种不同的通讯技术用来搭建无线网络；

每个边防哨所都要配备无线电收发器；有一些哨所还可以增配卫星电话。

任意两个配备了一条卫星电话线路的哨所（两边都有卫星电话）均可以通话，无论他们相距多远。而只通过无线电收发器通话的哨所之间的距离不能超过 D ，这是受收发器的功率限制。收发器的功率越高，通话距离 D 会更远，但同时价格也会更贵。

收发器需要统一购买和安装，所以全部哨所只能选择安装一种型号的收发器。换句话说，每一对哨所之间的通话距离都是同一个 D 。你的任务是确定收发器必须的最小通话距离 D ，使得每一对哨所之间至少有一条通话路径（直接的或者间接的）。

输入格式

第一行，2 个整数 S 和 P ， S 表示可安装的卫星电话的哨所数， P 表示边防哨所的数量。

接下里 P 行，每行两个整数 x, y 描述一个哨所的平面坐标 (x,y) ，以 km 为单位。

输出格式

第一行，1 个实数 D ，表示无线电收发器的最小传输距离，精确到小数点后两位。

输入输出样例 #1

输入 #1

```

1 2 4
2 0 100
3 0 300
4 0 600
5 150 750

```

输出 #1

```
1 | 212.13
```

说明/提示

数据范围及约定

- 对于 20% 的数据： $P=2$ ， $S=1$ ；
- 对于另外 20% 的数据： $P=4$ ， $S=2$ ；
- 对于 100% 的数据保证： $1 \leq S \leq 100$ ， $S < P \leq 500$ ， $0 \leq x, y \leq 10000$ 。

这是一个经典的最小生成树（MST）变种问题。我们可以通过 Kruskal 算法来解决。

解题思路

1. 模型转化：

- 假设有 P 个哨所。如果没有任何卫星电话（即 $S=0$ 或 1 ），我们需要将所有哨所连通，这就需要保留最小生成树中的所有 $P-1$ 条边，此时 D 为这 $P-1$ 条边中权值最大的那条边。
- 现在我们有 S 个卫星电话，这意味着我们可以将 S 个哨所配备卫星电话，从而将它们直接连通。换句话说，我们可以将整个图分成最多 S 个连通分量，每个连通分量内部通过无线电（距离不超过 D ）通信，而这 S 个连通分量之间可以通过卫星电话相互连接。
- 因此，我们的目标是通过无线电建立边，使得最终的连通分量个数减少到 S 个。

2. 具体步骤：

- 计算出所有哨所两两之间的欧几里得距离，作为边的权值。
- 将所有的边按照权值从小到大进行排序。
- 使用并查集（DSU）来维护连通性。初始时有 P 个独立的连通分量。
- 依次遍历排好序的边，如果边连接的两个哨所属于不同的连通分量，则合并它们，同时连通分量个数减 1。
- 当连通分量个数减少到 S 时停止。此时最后加入的那条边的长度就是所需的最小通话距离 D （即我们需要进行的合并次数为 $P-S$ 次，第 $P-S$ 次合并时的边权即为答案）。

Python 实现代码

```
1 import math
2 import sys
3
4
5 def solve():
6     # 读取所有输入
7     input_data = sys.stdin.read().split()
8     if not input_data:
9         return
10
11     S = int(input_data[0])
12     P = int(input_data[1])
13
14     coords = []
15     idx = 2
16     for _ in range(P):
17         x = float(input_data[idx])
18         y = float(input_data[idx + 1])
19         coords.append((x, y))
```

```

20     idx += 2
21
22     # 计算所有两两哨所之间的距离并存储为边
23     edges = []
24     for i in range(P):
25         for j in range(i + 1, P):
26             dx = coords[i][0] - coords[j][0]
27             dy = coords[i][1] - coords[j][1]
28             dist = math.sqrt(dx * dx + dy * dy)
29             edges.append((dist, i, j))
30
31     # 按边权（距离）升序排序
32     edges.sort(key=lambda x: x[0])
33
34     # 并查集初始化
35     parent = list(range(P))
36
37     def find(i):
38         path = []
39         while parent[i] != i:
40             path.append(i)
41             i = parent[i]
42         for node in path:
43             parent[node] = i
44         return i
45
46     def union(i, j):
47         root_i = find(i)
48         root_j = find(j)
49         if root_i != root_j:
50             parent[root_i] = root_j
51             return True
52         return False
53
54     count = 0
55     target = P - S # 需要进行的合并次数
56     ans = 0.0
57
58     # 如果 S >= P, 说明每个哨所都可以配一个卫星电话, D 理论上为 0
59     if target <= 0:
60         print("0.00")
61         return
62
63     # 运行 Kruskal 算法
64     for dist, u, v in edges:
65         if union(u, v):
66             count += 1
67             ans = dist
68             if count == target:
69                 break
70
71     # 输出结果, 保留两位小数
72     print(f"{ans:.2f}")
73
74
75 if __name__ == "__main__":
76     solve()

```

复杂度分析

- **时间复杂度：**
 - 计算所有边：共有 $P(P-1)/2$ 条边。对于 $P \leq 500$ ，边数约为 125,000。
 - 排序边： $O(E \log E)$ ，其中 $E \approx 125,000$ ，排序操作在 Python 中通常可以在 0.1 秒内完成。
 - 并查集操作：接近 $O(E \alpha(P))$ ，效率极高。
 - 整体时间复杂度为 $O(P^2 \log P)$ ，完全满足时限要求。
- **空间复杂度：**
 - 存储边需要 $O(P^2)$ 的空间，存储坐标和并查集需要 $O(P)$ 的空间，空间消耗在数兆字节以内，远低于空间限制。

E3633.最早完成陆地和水上游乐设施的时间 I

greedy, <https://leetcode.cn/problems/earliest-finish-time-for-land-and-water-rides-i/>

给你两种类别的游乐园项目：**陆地游乐设施** 和 **水上游乐设施**。

- **陆地游乐设施**
 - `landStartTime[i]` - 第 `i` 个陆地游乐设施最早可以开始的时间。
 - `landDuration[i]` - 第 `i` 个陆地游乐设施持续的时间。
- **水上游乐设施**
 - `waterStartTime[j]` - 第 `j` 个水上游乐设施最早可以开始的时间。
 - `waterDuration[j]` - 第 `j` 个水上游乐设施持续的时间。

一位游客必须从 **每个** 类别中体验 **恰好**一个**** 游乐设施，顺序 **不限**。

- 游乐设施可以在其开放时间开始，或 **之后任意时间** 开始。
- 如果一个游乐设施在时间 `t` 开始，它将在时间 `t + duration` 结束。
- 完成一个游乐设施后，游客可以立即乘坐另一个（如果它已经开放），或者等待它开放。

返回游客完成这两个游乐设施的 **最早可能时间**。

示例 1:

输入：landStartTime = [2,8], landDuration = [4,1], waterStartTime = [6], waterDuration = [3]

输出：9

解释：

- 方案 A（陆地游乐设施 0 → 水上游乐设施 0）：
 - 在时间 `landStartTime[0] = 2` 开始陆地游乐设施 0。在 `2 + landDuration[0] = 6` 结束。
 - 水上游乐设施 0 在时间 `waterStartTime[0] = 6` 开放。立即在时间 6 开始，在 `6 + waterDuration[0] = 9` 结束。
- 方案 B（水上游乐设施 0 → 陆地游乐设施 1）：
 - 在时间 `waterStartTime[0] = 6` 开始水上游乐设施 0。在 `6 + waterDuration[0] = 9` 结束。
 - 陆地游乐设施 1 在 `landStartTime[1] = 8` 开放。在时间 9 开始，在 `9 + landDuration[1] = 10` 结束。
- 方案 C（陆地游乐设施 1 → 水上游乐设施 0）：
 - 在时间 `landStartTime[1] = 8` 开始陆地游乐设施 1。在 `8 + landDuration[1] = 9` 结束。

- 水上游乐设施 0 在 `waterStartTime[0] = 6` 开放。在时间 9 开始，在 `9 + waterDuration[0] = 12` 结束。
- 方案 D（水上游乐设施 0 → 陆地游乐设施 0）：
 - 在时间 `waterStartTime[0] = 6` 开始水上游乐设施 0。在 `6 + waterDuration[0] = 9` 结束。
 - 陆地游乐设施 0 在 `landStartTime[0] = 2` 开放。在时间 9 开始，在 `9 + landDuration[0] = 13` 结束。

方案 A 提供了最早的结束时间 9。

示例 2:

输入: `landStartTime = [5], landDuration = [3], waterStartTime = [1], waterDuration = [10]`

输出: 14

解释:

- 方案 A（水上游乐设施 0 → 陆地游乐设施 0）：
 - 在时间 `waterStartTime[0] = 1` 开始水上游乐设施 0。在 `1 + waterDuration[0] = 11` 结束。
 - 陆地游乐设施 0 在 `landStartTime[0] = 5` 开放。立即在时间 11 开始，在 `11 + landDuration[0] = 14` 结束。
- 方案 B（陆地游乐设施 0 → 水上游乐设施 0）：
 - 在时间 `landStartTime[0] = 5` 开始陆地游乐设施 0。在 `5 + landDuration[0] = 8` 结束。
 - 水上游乐设施 0 在 `waterStartTime[0] = 1` 开放。立即在时间 8 开始，在 `8 + waterDuration[0] = 18` 结束。

方案 A 提供了最早的结束时间 14。

提示:

- `1 <= n, m <= 100`
- `landStartTime.length == landDuration.length == n`
- `waterStartTime.length == waterDuration.length == m`
- `1 <= landStartTime[i], landDuration[i], waterStartTime[j], waterDuration[j] <= 1000`

一个可行且高效的方法是遍历所有可能的陆地游乐设施和水上游乐设施的组合，分别计算“先体验陆地项目再体验水上项目”以及“先体验水上项目再体验陆地项目”的结束时间，并求出全局最小值。

算法步骤

对于每一个陆地游乐设施 i （其开始时间为 `landStartTime[i]`，持续时间为 `landDuration[i]`）和水上游乐设施 j （其开始时间为 `waterStartTime[j]`，持续时间为 `waterDuration[j]`）：

1. 先进行陆地游乐设施 i ，后进行水上游乐设施 j ：

- 陆地项目的结束时间为 $E_{land} = \text{landStartTime}[i] + \text{landDuration}[i]$ 。
- 随后，水上项目最早可以在 $\max(E_{land}, \text{waterStartTime}[j])$ 开始，其完成时间为 $\max(E_{land}, \text{waterStartTime}[j]) + \text{waterDuration}[j]$ 。

2. 先进行水上游乐设施 j ，后进行陆地游乐设施 i ：

- 水上项目的结束时间为 $E_{water} = \text{waterStartTime}[j] + \text{waterDuration}[j]$ 。
- 随后，陆地项目最早可以在 $\max(E_{water}, \text{landStartTime}[i])$ 开始，其完成时间为 $\max(E_{water}, \text{landStartTime}[i]) + \text{landDuration}[i]$ 。

通过遍历所有 (i, j) 对 (其中 $i \in [0, n - 1]$, $j \in [0, m - 1]$) , 并维护以上两种方案的最小值, 即可求得最早完成两个游乐设施的时间。

Python 代码实现

```
1  from typing import List
2
3  class Solution:
4      def earliestFinishTime(self, landStartTime: List[int], landDuration: List[int],
5      waterStartTime: List[int], waterDuration: List[int]) -> int:
6          n = len(landStartTime)
7          m = len(waterStartTime)
8          min_finish_time = float('inf')
9
10         for i in range(n):
11             lst = landStartTime[i]
12             ldu = landDuration[i]
13             land_end = lst + ldu
14
15             for j in range(m):
16                 wst = waterStartTime[j]
17                 wdu = waterDuration[j]
18                 water_end = wst + wdu
19
20                 # 方案一: 先陆地, 后水上
21                 finish_1 = max(land_end, wst) + wdu
22
23                 # 方案二: 先水上, 后陆地
24                 finish_2 = max(water_end, lst) + ldu
25
26                 # 更新全局最小值
27                 if finish_1 < min_finish_time:
28                     min_finish_time = finish_1
29                 if finish_2 < min_finish_time:
30                     min_finish_time = finish_2
31
32         return min_finish_time
```

复杂度分析

- **时间复杂度:** $O(n \times m)$ 。其中 n 为陆地游乐设施的数量, m 为水上游乐设施的数量。由于题目中 $n, m \leq 100$, 最大循环次数约为 10^4 , 能够在极短时间内运行完毕。
- **空间复杂度:** $O(1)$ 。仅使用了若干变量来记录状态, 不需要额外的辅助存储空间。